

第 05 讲

Linear Transformation 与 $y = Wx + b$: Weight、Bias 到底是什么

本讲目标：把上一讲的矩阵乘法正式装进神经网络的一层里。通过一个 $3 \rightarrow 2$ 的小例子，逐个数字理解输入 x 、权重 W 、偏置 b 、输出 y 、Shape 与参数数量，并对齐 NumPy 与 PyTorch 的写法。

讲义完整版

目录

1 为什么这一讲是真正进入“神经网络”的入口	3
1.1 本讲最低验收标准	3
2 先从一个输出开始：一个“神经元”在做什么	3
2.1 【原理】加权求和	4
2.2 【通俗解释】调音台	4
3 从一个输出扩展到两个输出：为什么需要矩阵 W	4
4 【具体数字例子】完整手算一个 $3 \rightarrow 2$ Linear Layer	5
4.1 第一个输出	5
4.2 第二个输出	5
5 为什么需要 Bias?	6
5.1 【通俗解释】Weight 管“方向和混合”，Bias 管“整体平移”	6
6 Batch 情况：很多样本共享同一组参数	7
7 Bias Broadcasting：为什么 $(B, D_{out}) + (D_{out},)$ 可以相加?	7
8 Parameter：什么叫“模型参数”?	8
9 Weight 和 Bias 是怎样来的?	8
10 一个重要记法陷阱： $y = Wx + b$ 与 $x @ W + b$	9
10.1 数学教材常用列向量约定	9
10.2 NumPy / 数据表思维常用行向量约定	9
11 PyTorch 的 nn.Linear 到底做什么?	9
12 【代码实验】先 NumPy，后 PyTorch 验证	10
12.1 NumPy 完整最小代码	10
12.2 PyTorch 对照	10
13 常见误区	11
13.1 误区 1：Weight 就是输入的“重要程度”	11
13.2 误区 2：Bias 是可有可无的“误差项”	11

13.3 误区 3: Bias 每个输入都有一个	11
13.4 误区 4: Linear Layer 只有一个神经元	11
13.5 误区 5: Shape 能乘上就一定语义正确	11
14 与 Transformer / LLM 的直接连接	11
14.1 Q/K/V Projection	11
14.2 Transformer FFN / MLP	12
14.3 Vocabulary Projection	12
15 本讲总结: 把一层神经网络压缩成 6 个问题	12
16 下一讲预告	13

1 为什么这一讲是真正进入“神经网络”的入口

前四讲我们主要在学习“数字怎样组织、怎样相乘”。从这一讲开始，这些数学不再是孤立知识，而会变成神经网络真正执行的计算。

一个最基本的神经网络层，核心通常可以写成：

$$y = Wx + b$$

或者在批量数据的 NumPy 写法中常见：

$$Y = XW + b$$

本讲最重要的一句话

Weight 决定“输入的哪些成分，以多大力度，组合成哪些输出”；Bias 决定“在这个组合结果上再平移多少”。

与 LLM / Transformer / Agent Systems 的连接

你以前在 LLM 课程中已经见过 $Q = XW_Q$ 、 $K = XW_K$ 、 $V = XW_V$ 。本讲学完后，你应该意识到：它们就是三组不同的 Weight 对同一份 hidden states 做线性投影。以后 Transformer FFN、Attention projection、最终 vocabulary projection 都会反复使用这一结构。

1.1 本讲最低验收标准

学完后你应该能做到：

1. 解释 x 、 W 、 b 、 y 分别是什么；
2. 根据 D_{in} 和 D_{out} 写出 Weight 与 Bias 的 Shape；
3. 手算一个 $3 \rightarrow 2$ 的线性层；
4. 解释 Weight 的一列为什么对应一个输出特征；
5. 解释 Bias 为什么是“每个输出一个”；
6. 计算一层的参数数量；
7. 理解一个 Batch 如何共享同一组 W 和 b ；
8. 看懂 NumPy 的 $X @ W + b$ ；
9. 看懂 PyTorch `nn.Linear(in,out)` 内部 weight 的 Shape；
10. 知道严格数学上 $Wx + b$ ($b \neq 0$) 属于 affine transformation，而深度学习 API 仍习惯叫 Linear layer。

2 先从一个输出开始：一个“神经元”在做什么

2.1 【原理】加权求和

假设一个样本有 3 个输入特征：

$$x = [x_1, x_2, x_3]$$

我们希望得到一个输出 z 。最简单的做法是给每个输入一个权重：

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

其中：

- x_i ：输入数字；
- w_i ：对应输入的 Weight；
- b ：Bias；
- z ：当前神经元输出的一个数字。

2.2 【通俗解释】调音台

可以把输入想成三个音轨：人声、鼓、吉他。Weight 就像每个音轨的音量旋钮：有的放大、有的减弱、有的甚至反向。最后把它们混合起来，再整体加一个基线偏移 Bias，就得到一个输出通道。

不要把 Weight 过度解释成“重要性”

在非常简单的线性模型中，Weight 大小有时可以帮助解释影响方向。但在深层神经网络中，一个参数往往与成千上万个其他参数共同作用，不能简单说“这个 weight 大，所以这个输入最重要”。科研中做解释需要更严格的方法。

3 从一个输出扩展到两个输出：为什么需要矩阵 W

现在输入仍然有 3 个数字，但我们想同时得到 2 个输出：

$$y = [y_1, y_2]$$

那么需要两套不同的权重。

采用本课程 NumPy 的行向量约定：

$$x : (3,), \quad W : (3, 2), \quad b : (2,)$$

其中：

$$W = \begin{bmatrix} 0.5 & -1.0 \\ 2.0 & 0.0 \\ -0.5 & 1.5 \end{bmatrix}$$

第一列：

$$\begin{bmatrix} 0.5 \\ 2.0 \\ -0.5 \end{bmatrix}$$

专门用来生成第 1 个输出；第二列专门生成第 2 个输出。

Weight Matrix 的 Shape 直觉

在本课程主要使用的 $X @ W$ 记法里：

$$W : (D_{in}, D_{out})$$

行数 = 输入维度；列数 = 输出维度。所以每一个输出维度，对应 W 的一列。

4 【具体数字例子】完整手算一个 $3 \rightarrow 2$ Linear Layer

设：

$$x = [2, -1, 3]$$

$$W = \begin{bmatrix} 0.5 & -1.0 \\ 2.0 & 0.0 \\ -0.5 & 1.5 \end{bmatrix}, \quad b = [0.2, -0.3]$$

先不加 Bias：

$$xW = [2, -1, 3] \begin{bmatrix} 0.5 & -1.0 \\ 2.0 & 0.0 \\ -0.5 & 1.5 \end{bmatrix}$$

输出 Shape 为：

$$(3,) @ (3, 2) \rightarrow (2,)$$

4.1 第一个输出

$$\begin{aligned} y_1^{\text{pre-bias}} &= 2 \times 0.5 + (-1) \times 2.0 + 3 \times (-0.5) \\ &= 1 - 2 - 1.5 = -2.5 \end{aligned}$$

再加第一个 Bias：

$$y_1 = -2.5 + 0.2 = -2.3$$

4.2 第二个输出

$$y_2^{\text{pre-bias}} = 2 \times (-1.0) + (-1) \times 0 + 3 \times 1.5$$

$$= -2 + 0 + 4.5 = 2.5$$

再加 Bias:

$$y_2 = 2.5 - 0.3 = 2.2$$

所以:

$$y = [-2.3, 2.2]$$

每一个数字到底来自哪里

输出 y_1 并不是“凭空产生”的。它来自输入 $[2, -1, 3]$ 与 Weight 第一列做 Dot Product, 再加 b_1 。 y_2 同理, 只是换成 Weight 第二列和 b_2 。

5 为什么需要 Bias?

如果没有 Bias:

$$y = xW$$

当 $x = 0$ 时, 必然有:

$$y = 0$$

这意味着映射被强制经过原点。

加入 Bias:

$$y = xW + b$$

即使输入为零, 也可以得到:

$$y = b$$

5.1 【通俗解释】Weight 管“方向和混合”, Bias 管“整体平移”

想象一条直线:

$$y = wx + b$$

w 改变斜率, b 把整条线向上或向下移动。在多维情况下, 直觉类似: Weight 改变输入如何组合与投影; Bias 让结果不必被限制在通过原点的位置。

严格术语: Linear 还是 Affine?

严格线性代数中, 纯粹的线性变换是:

$$y = Wx$$

而:

$$y = Wx + b$$

若 $b \neq 0$, 更准确叫 **Affine Transformation (仿射变换)**。但深度学习框架通常仍把带 Bias 的层命名为 Linear。例如 PyTorch 的 `nn.Linear` 默认包含 Bias。

6 Batch 情况：很多样本共享同一组参数

假设现在有 3 个样本，每个样本都有 3 个输入特征：

$$X : (3, 3)$$

第一维是 Batch 中的样本数，第二维是特征数。

Weight 仍然是：

$$W : (3, 2)$$

则：

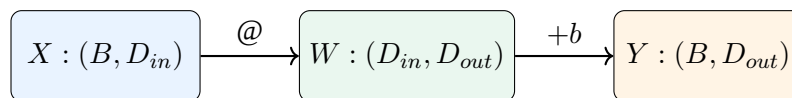
$$XW : (3, 2)$$

再加：

$$b : (2,)$$

得到：

$$Y : (3, 2)$$



共享参数是神经网络的基本事实

Batch 中的每个样本并不是拥有自己的一套 Weight。所有样本共享同一个 W 和 b 。训练时我们学习的是这套共享参数。

7 Bias Broadcasting：为什么 $(B, D_{out}) + (D_{out},)$ 可以相加？

例如：

$$XW : (3, 2), \quad b : (2,)$$

NumPy 会把 $b = [b_1, b_2]$ 视为要加到每一行：

$$\begin{bmatrix} a & b \\ c & d \\ e & f \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \end{bmatrix}$$

等价于：

$$\begin{bmatrix} a + b_1 & b + b_2 \\ c + b_1 & d + b_2 \\ e + b_1 & f + b_2 \end{bmatrix}$$

这叫 Broadcasting。

常见误区 / 风险

Broadcasting 很方便，但也可能悄悄掩盖 Shape 错误。未来调试时，某些“不报错但结果不对”的问题，恰恰来自意外 Broadcasting。

8 Parameter：什么叫“模型参数”？

Weight 和 Bias 都属于 **learnable parameters**（可学习参数）。

对于一个 $D_{in} \rightarrow D_{out}$ 的 Linear Layer：

$$\#W = D_{in} \times D_{out}$$

$$\#b = D_{out}$$

所以总参数数：

$$D_{in}D_{out} + D_{out}$$

例如 $3 \rightarrow 2$ ：

$$3 \times 2 + 2 = 8$$

一共有 8 个可学习数字。

连接 LLM 中“几十亿参数”

“7B 模型”里的 parameter 并不是 70 亿个神秘模块，而大体就是分散在大量矩阵、Embedding table、Normalization 参数等结构里的几十亿个可学习数字。Linear projection 的 Weight Matrix 通常就是参数大户之一。

9 Weight 和 Bias 是怎样来的？

刚创建模型时，Weight 通常先被初始化为某些小随机值，Bias 常初始化为零或其他简单值。之后训练不断执行：

$$\text{Forward} \rightarrow \text{Loss} \rightarrow \text{Gradient} \rightarrow \text{Update}$$

于是 Weight 与 Bias 慢慢改变。

核心概念

Weight/Bias 不是程序员手工填出最终答案。程序员主要决定结构和训练方法；具体参数值由训练数据和优化过程逐步学习得到。

后面的 Gradient、Backpropagation、Optimizer 课程会回答：“模型究竟怎么知道应该把哪个 Weight 调大、哪个调小？”

10 一个重要记法陷阱： $y = Wx + b$ 与 $x @ W + b$

你以后一定会同时看到两种记法。

10.1 数学教材常用列向量约定

把输入写成列向量：

$$x : (D_{in}, 1)$$

则：

$$W : (D_{out}, D_{in})$$

所以：

$$Wx : (D_{out}, 1)$$

即：

$$y = Wx + b$$

10.2 NumPy / 数据表思维常用行向量约定

把一个样本写成一行：

$$x : (D_{in},)$$

或 Batch：

$$X : (B, D_{in})$$

常把 Weight 写成：

$$W : (D_{in}, D_{out})$$

于是：

$$Y = XW + b$$

不要争论哪个“才是对的”

两种写法表达的是同一类线性/仿射映射，只是向量朝向和 Weight 的存储方向不同。读公式时永远先看 Shape，而不是死背 W 在左还是在右。

11 PyTorch 的 nn.Linear 到底做什么？

当我们写：

```
layer = nn.Linear(3, 2)
```

意思是：

- 输入最后一维长度 3;
- 输出最后一维长度 2;
- 创建一个 Weight;
- 默认还创建一个 Bias。

但 PyTorch 内部:

```
layer.weight.shape = (2, 3)
```

即:

$$(D_{out}, D_{in})$$

而前向在行向量 Batch 视角等价于:

$$Y = XW^T + b$$

PyTorch 初学者高频困惑

如果你刚学会 $X : (B, 3)$ 与 $W : (3, 2)$ 做 $X @ W$, 然后发现 `nn.Linear(3, 2).weight.shape` 居然是 $(2, 3)$, 不要认为前面的数学错了。PyTorch 只是选择把 Weight 转置方向存储, 计算时使用它的转置。

12 【代码实验】先 NumPy, 后 PyTorch 验证

12.1 NumPy 完整最小代码

```
import numpy as np

x = np.array([2.0, -1.0, 3.0])
W = np.array([[0.5, -1.0],
              [2.0, 0.0],
              [-0.5, 1.5]])
b = np.array([0.2, -0.3])

y = x @ W + b

print("x.shape =", x.shape)    # (3,)
print("W.shape =", W.shape)    # (3, 2)
print("b.shape =", b.shape)    # (2,)
print("y =", y)                # [-2.3  2.2]
print("y.shape =", y.shape)    # (2,)
```

12.2 PyTorch 对照

```
import torch
```

```
import torch.nn as nn

layer = nn.Linear(3, 2)
print(layer.weight.shape) # torch.Size([2, 3])
print(layer.bias.shape)   # torch.Size([2])
```

本讲实验脚本会把 NumPy 的 W 转置后复制给 PyTorch，再验证两边输出完全一致。

13 常见误区

13.1 误区 1: Weight 就是输入的“重要程度”

不准确。Weight 首先只是计算参数。深层模型中的语义是多层共同作用的结果。

13.2 误区 2: Bias 是可有可无的“误差项”

不是。Bias 不是 prediction error。它是模型主动学习的参数，用于平移映射。

13.3 误区 3: Bias 每个输入都有一个

在普通 Linear Layer 中，通常是**每个输出特征一个 Bias**，所以 $b : (D_{out},)$ 。

13.4 误区 4: Linear Layer 只有一个神经元

若输出维度是 4096，就可以把它理解成同时计算 4096 个输出单元，每个输出都有一组输入权重。

13.5 误区 5: Shape 能乘上就一定语义正确

错误。Shape 只是必要条件。把错误含义的数据维度凑到可相乘，并不会让模型设计变正确。

14 与 Transformer / LLM 的直接连接

14.1 Q/K/V Projection

如果 hidden states:

$$X : (B, T, D)$$

并有:

$$W_Q : (D, d_q)$$

则最后一维发生:

$$D \rightarrow d_q$$

输出：

$$Q : (B, T, d_q)$$

Batch 轴和 Token 轴保留下来，Linear Layer 主要作用于每个 Token 的最后一维表示。

14.2 Transformer FFN / MLP

典型结构会把 hidden dimension 从 D 扩大到更高维，再压回来，例如：

$$D \rightarrow 4D \rightarrow D$$

其中两次核心操作都是 Linear Layer，只是中间加入 Activation Function。后面我们会专门学习为什么没有 Activation 的多层 Linear 最终仍等价于一个 Linear。

14.3 Vocabulary Projection

模型最后需要把 hidden state 变成词表中每个 Token 的分数。若：

$$h : (D), \quad W_{vocab} : (D, V)$$

则：

$$hW_{vocab} : (V)$$

得到 V 个 vocabulary logits。

科研阅读时怎样看一层 Linear

看到论文写 “project from 768 to 256 dimensions”，先把自然语言翻译成： $D_{in} = 768$ ， $D_{out} = 256$ 。然后问：Weight Shape 是什么？参数数是多少？输出 Shape 哪一维改变？如果加了 Bias，Bias 有几个参数？这比死记 API 更可靠。

15 本讲总结：把一层神经网络压缩成 6 个问题

以后看到任意 Linear Layer，按顺序问：

1. 输入是什么？
2. 输入最后一维是多少？
3. 输出最后一维要变成多少？
4. Weight 的 Shape 是什么？
5. Bias 的 Shape 是什么？
6. 输出 Shape 是什么？

本节检查点

最低闭环：给你 $X : (32, 768)$ ，要映射到 3072 维，你应该立即回答：采用 $X@W + b$ 记法时， $W : (768, 3072)$ 、 $b : (3072,)$ 、输出 $(32, 3072)$ ，参数数为 $768 \times 3072 + 3072$ 。

16 下一讲预告

下一讲进入第一阶段最后一讲：**Batch 与批量矩阵运算**。我们会把当前二维 $X : (B, D)$ 扩展到 LLM 中更常见的三维：

$$X : (B, T, D)$$

真正解释为什么一个 Linear Layer 能同时作用于一个 Batch 中所有 Token，而不需要手写两层循环。